**ASSIGNMENT 15.2 AI ASSISTED CODING 2303A52490**

## Task Descriptor-1: Setup Fast API Backend

```
user@ubuntu:~/lab15$ python3 -m venv venv
user@ubuntu:~/lab15$ source venv/bin/activate
(venv) user@ubuntu:~/lab15$ pip install fastapi uvicorn
Collecting fastapi
  Downloading fastapi-0.115.0-py3-none-any.whl (92 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 92.8/92.8 kB 1.2 MB/s eta
0:00:00
Collecting uvicorn
  Downloading uvicorn-0.32.0-py3-none-any.whl (85 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.6/85.6 kB 1.4 MB/s eta
0:00:00
...
Successfully installed anyio-4.6.0 fastapi-0.115.0 ...
```

Used AI prompt: "Generate a basic FastAPI app with root endpoint returning welcome message."

Created main.py:

```
from fastapi import FastAPI

app = FastAPI(title="AI Powered Store API", version="1.0.0")

@app.get("/")
def read_root():
    """Root endpoint - Welcome message for the API."""
    return {"message": "Welcome to AI-powered FastAPI"}
```

Ran the server.

```
(venv) user@ubuntu:~/lab15$ uvicorn main:app --reload --host 127.0.0.1 --port 8000
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Tested with curl:

```
$ curl http://127.0.0.1:8000/
{"message":"Welcome to AI-powered FastAPI"}
```

Accessed http://127.0.0.1:8000/docs - Swagger UI loaded perfectly with auto-docs.

## Task Descriptor-2: API Create and Read Items

Extended the code with items list and CRUD read/create. AI prompt: "Add GET /items and POST /items to FastAPI app using list for storage."

Updated main.py:

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List

app = FastAPI(title="AI Powered Store API", version="1.0.0")

items_db: List[dict] = []

class Item(BaseModel):
    name: str
    price: float

@app.get("/")
def read_root():
    """Root endpoint - Welcome message."""
    return {"message": "Welcome to AI-powered FastAPI"}

@app.get("/items", response_model=List[Item])
def get_items():
    """Retrieve all items from the store."""
    return items_db

@app.post("/items", response_model=dict)
def add_item(item: Item):
    """Create a new item in the store."""
    items_db.append(item.dict())
    return {"message": "Item created", "item": item}
```

Restarted server.

Test GET:

```
$ curl http://127.0.0.1:8000/items
[]
```

Test POST:

```
$ curl -X POST "http://127.0.0.1:8000/items" -H "Content-Type: application/json" -d '{"name":"Pen","price":20}'
{"message":"Item created","item":{"name":"Pen","price":20.0}}
```

GET now shows:

```
$ curl http://127.0.0.1:8000/items
[{"name":"Pen","price":20.0}]
```

## Task Descriptor-3: Update Existing Item

AI prompt: "Add PUT /items/{index} endpoint to update item by index in FastAPI."

Added to main.py:

```
@app.put("/items/{index}", response_model=dict)
def update_item(index: int, item: Item):
    """Update an existing item by its index."""
    if index >= len(items_db):
        return {"error": "Item not found"}
    items_db[index] = item.dict()
    return {"message": "Item updated", "item": item}
```

Test update (after adding Pen):

```
$ curl -X PUT "http://127.0.0.1:8000/items/0" -H "Content-Type: application/json" -d '{"name":"Pencil","price":10}'
{"message":"Item updated","item":{"name":"Pencil","price":10.0}}
```

Verified:

```
$ curl http://127.0.0.1:8000/items
[{"name":"Pencil","price":10.0}]
```

Test invalid index:

```
$ curl -X PUT "http://127.0.0.1:8000/items/5" -H "Content-Type: application/json" -d '{"name":"Test","price":5}'
{"error":"Item not found"}
```

## Task Descriptor-4: Delete Item

AI prompt: "Implement DELETE /items/{index} for FastAPI item list."

Added:

```
@app.delete("/items/{index}", response_model=dict)
def delete_item(index: int):
    """Delete an item by its index."""
    if index >= len(items_db):
        return {"error": "Item not found"}
    removed = items_db.pop(index)
    return {"message": "Item removed", "item": removed}
```

Added another item first:

```
$ curl -X POST "http://127.0.0.1:8000/items" -H "Content-Type: application/json" -d '{"name":"Notebook","price":50}'
{"message":"Item created","item":{"name":"Notebook","price":50.0}}
```

List: [{"name":"Pencil","price":10.0},{"name":"Notebook","price":50.0}]

Delete first:

```
$ curl -X DELETE "http://127.0.0.1:8000/items/0"
{"message":"Item removed","item":{"name":"Pencil","price":10.0}}
```

List now: [{"name":"Notebook","price":50.0}]

Invalid delete:

```
$ curl -X DELETE "http://127.0.0.1:8000/items/10"
{"error":"Item not found"}
```

## Task Descriptor-5: Add Auto-Generated Documentation

Enhanced all endpoints with docstrings and descriptions. FastAPI auto-generates Swagger/ReDoc docs.

Final main.py (complete code):

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List
```

```
app = FastAPI(
    title="AI Powered Store API",
    version="1.0.0",
    description="RESTful API for store items with full CRUD operations generated with AI assistance."
)

items_db: List[dict] = []

class Item(BaseModel):
    name: str
    price: float

@app.get("/", summary="Root Endpoint", description="Basic welcome message for the API.")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}

@app.get("/items", response_model=List[Item], summary="Get All Items", tags=["items"])
def get_items():
    """Retrieve the list of all items in the store database."""
    return items_db

@app.post("/items", response_model=dict, summary="Create Item", tags=["items"])
def add_item(item: Item):
    """Add a new item to the store. Returns the created item."""
    items_db.append(item.dict())
    return {"message": "Item created", "item": item}

@app.put("/items/{index}", response_model=dict, summary="Update Item", tags=["items"])
def update_item(index: int, item: Item):
    """Update item at specified index. Returns updated item or error."""
    if index >= len(items_db):
        return {"error": "Item not found"}
    items_db[index] = item.dict()
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}", response_model=dict, summary="Delete Item", tags=["items"])
def delete_item(index: int):
    """Remove item at specified index. Returns removed item or error."""
    if index >= len(items_db):
        return {"error": "Item not found"}
    removed = items_db.pop(index)
    return {"message": "Item removed", "item": removed}
```